УНИВЕРЗИТЕТ
У НОВОМ САДУ

ФАКУЛТЕТ
ТЕХНИЧКИХ НАУКА

Трг Доситеја Обрадовића 6, 21000 Нови Сад, Република Србија
Деканат: 021 6350-413; 021 450-810; Централа: 021 485 2000
Рачуноводство: 021 458-220; Студентска служба: 021 6350-763
Телефакс: 021 458-133; e-mail: ftndean@uns.ac.rs

ИНТЕГРИСАНИ
СИСТЕМ
МЕНАЏМЕНТА
СЕРТИФИКОВАН ОД:

Embeded operativni sistemi

NAZIV PROJEKTA:

Razvoj softverske podrške za sistem za detekciju lica upotrebom Viola-Jones metodologije

PROJEKAT IZRADILI:

Radivoje Jović EE 202/2021
Vojislav Sandić
Sara Glišević

Sadržaj

Uvod 4

Arhitektura sistema 4

Pregled hardverske arhitekture 4

Magistralni interfejsi 5

Adresna mapa 5

Registarska mapa 6

Ograničenja integracije 6

Tok podataka kroz sistem 7

Linux kernel drajver 9

Uloga drajvera u sistemu 9

Platform drajver i stablo uređaja (device tree) 9

Prepoznavanje uređaja u probe funkciji 10

Uloga device fajlova 10

Upis u kontrolni uređaj 11

Prekidna rutina 13

Čitanje iz kontrolnog uređaja 14

Upis redova 15

Čitanje detekcija 16

Uklanjanje drajvera 17

Korisnička aplikacija 18

Način pokretanja 18

Priprema slike 18

Integralna i kvadrirana integralna slika 19

Pomjeranje prozora po y 20

Pokretanje sweep-a i čekanje 21

Grupisanje detekcija 21

Pokretanje sistema i rezultati 23

Kompajliranje i učitavanje drajvera 23

Rezultati detekcije 23

Analiza izmjerenih veličina 24
Zaključak 25
Literatura 26

Uvod

Ovaj dokument opisuje razvoj sistema za detekciju lica sa hardverskom akceleracijom na Zybo Z7-10 razvojnoj ploči sa Xilinx Zynq-7000 SoC čipom. Detekcija se zasniva na Viola-Jones metodi, a vremenski kritičan dio algoritma - prolazak kroz kaskadu od 2913 slabih klasifikatora - implementiran je kao IP jezgro u VHDL jeziku koji se izvršava u programabilnoj logici, dok procesorski sistem pod Linux operativnim sistemom upravlja tokom obrade i pripremom podataka.

Motivacija za hardversku akceleraciju je priroda samog algoritma. Za jednu poziciju detekcijskog prozora potrebno je pročitati i sabrati stotine pravougaonika iz integralne slike, a jedna slika sadrži desetine hiljada pozicija prozora kroz sve skale piramide. Softverska implementacija na procesoru opšte namjene troši veliki broj instrukcija po svakom pravougaoniku, dok hardverska implementacija istu operaciju obavlja u dva taktna ciklusa, uz memoriju organizovanu tako da sva četiri ćoška pravougaonika stižu istovremeno.

Sistem je razvijen i testiran na Zybo Z7-10 ploči sa PetaLinux 2022.2 distribucijom i kernelom verzije 5.15.36. Referenca za provjeru ispravnosti bila je postojeća Vitis baremetal aplikacija koja koristi isti IP core bez operativnog sistema, čime je bilo moguće razdvojiti greške u hardveru od grešaka u softverskom sloju.

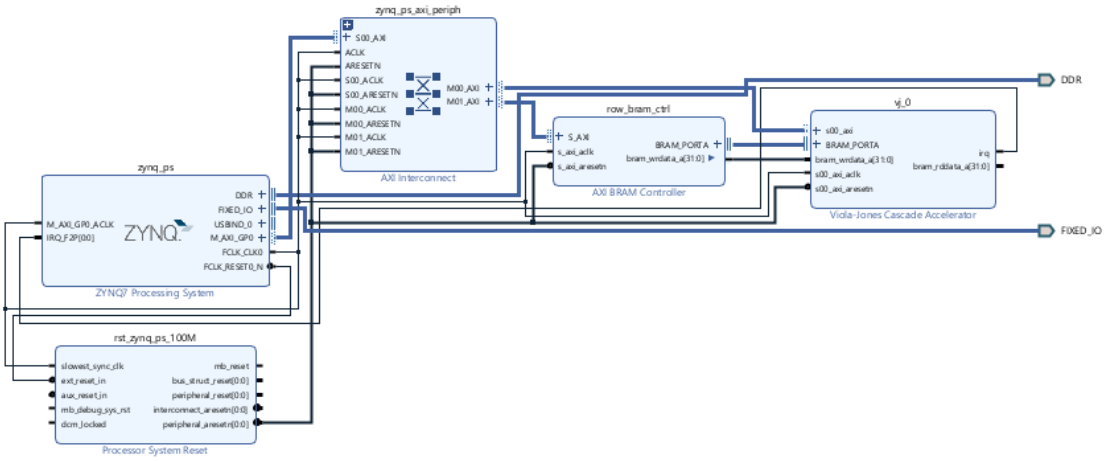
Arhitektura sistema

Pregled hardverske arhitekture

Zybo Z7-10 razvojna ploča bazirana je na Xilinx Zynq-7000 SoC čipu, koji integriše ARM Cortex-A9 procesor i programabilnu logiku Artix-7 familije. Procesorski sistem (PS) pokreće Linux operativni sistem zajedno sa kernel drajverom i korisničkom aplikacijom, dok programabilna logika (PL) sadrži Viola-Jones kaskadni detektor lica opisan u VHDL jeziku.

Ovakva podjela posla je za detekciju lica prirodna. Pošto čitava kaskada sadrži ukupno 2913 slabih klasifikatora, od kojih svaki zahtijeva čitanje dva ili tri pravougaonika iz integralne slike i njihovo težinsko sabiranje, za jednu poziciju prozora to je u najgorem slučaju nekoliko hiljada memorijskih pristupa, a jedna slika sadrži desetine hiljada pozicija prozora kroz sve skale. Softverska implementacija na procesoru opšte namjene troši veliki broj instrukcija po jednom pravougaoniku - računanje adrese, čitanja četiri ćoška, sabiranja i oduzimanja tih ćoškova - dok hardverska implementacija za istu operaciju troši svega nekoliko taktnih ciklusa, uz memoriju organizovanu tako da sva četiri ćoška pravougaonika stižu istovremeno.

Komunikacija između PS i PL odvija se kroz AXI magistralu. Za razliku od tipičnih sistema sa jednim AXI4-Lite interfejsom, ovaj sistem koristi dva odvojena magistralna puta, što je jedna od centralnih projektnih odluka i objašnjena je u narednom potpoglavlju.



Slika 1. Blok dizajn sistema sa prikazanim komponentama

Magistralni interfejsi

IP jezgro otkriva dva potpuno odvojena interfejsa prema procesorskom sistemu:

S00_AXI je AXI4-Lite interfejs koji kontrolne i statusne registre. Kroz njega procesor konfiguriše parametre pretrage, pokreće obradu i čita rezultate. Saobraćaj kroz ovaj interfejs je zanemarljiv i sastoji se iz nekoliko upisa i čitanja po jednoj poziciji reda.

BRAM_PORTA je interfejs kojim upravlja AXI BRAM kontroler i kroz koji se prenose redovi integralne slike u memoriju IP jezgra,š što je jedini prenos velikog obima u sistemu.

Razlog za razdvajanje je propusni opseg, pošto detekcija zahtijeva prenos ogromnog broja (reda miliona) riječi, koji AXI BRAM kontroler obavlja u burst transakcijama brzinom od približno jedne riječi po taktnom ciklusu, dok bi pojedinačni AXI4-Lite upisi - svaki sa punim adresnim i handshake ciklusom - trajali otprilike deset puta duže, pa bi tako prenos podataka o integralnim slikama kroz registarski interfejs postao dominantni trošak sistema i hardverska akceleracija bi izgubila poentu.Ova odluka ima direktnu posljedicu na softver koja se ogleda u tome da kernel drajver i aplikacija moraju tretirati registre i podatke o slici kao dva odvojena kanala, sa različitim interfejsima prema korisničkom prostoru, što je detaljnije opisano u jednom od narednih poglavlja.

Adresna mapa

U Vivado Address Editoru dodijeljene su sljedeće adrese:

Komponenta	Bazna adresa	Veličina	Namjena
vj_0 (S00_AXI)	0x4000_0000	4 KB	Kontrolni i statusni registri
row_bram_ctrl (S_AXI)	0x4200_0000	64 KB	Kontrola za row bank memoriju

Tabela 1. Adresna mapa sistema

Od 64 KB dodijeljenih BRAM kontroleru, sistem stvarno koristi 54 KB. Ostatak adresnog prostora se ne dekodira i predstavlja rezervu za eventualno proširenje broja banaka.Interrupt

izlaz IP jezgra (**irq**) vezan je na **IRQ_F2P[0]** ulaz procesorskog sistema, čime se završetak obrade signalizira prekidom umjesto softverskim pollingom.

Registarska mapa

Svi registri su 32-bitni i nalaze se na bajt ofsetima u odnosu na baznu adresu 0x4000_0000:

Offset	Naziv	Pristup	Opis
0x00	CTRL	RW	b0 RESET, b1 START, b2 MODE, b4 DONE
0x04	X_START	RW	Pozicija prozora x (mod A) / prvi x (mod B)
0x08	WIDTH	RW	Širina trenutne skale
0x0C	RESULT	RO	Mod A: 1 = lice, −i = odbijeno u fazi i
0x10	X_LAST	RW	Mod B: posljednji x koji se testira
0x14	X_STEP	RW	Mod B: korak pomjeranja prozora
0x18	HIT_COUNT	RO	Mod B: broj detekcija u hit baferu
0x1C	HIT_DATA	RO	Mod B: sadržaj hit bafera na HIT_ADDR
0x20	ROW_BASE	RW	Rotacija 25 banaka integralne slike, 0-24
0x24	STATUS	RO	b0 BUSY
0x28	HIT_ADDR	RW	Indeks u hit bafer za HIT_DATA

Tabela 2. Registarska mapa IP jezgra

Bitovi **RESET** i **START** u **CTRL** registru su **self-clearing** - hardver ih automatski spušta nakon jednog taktnog ciklusa, pa softver ne mora ručno vraćati registar na nulu. Bit **DONE** je suprotan slučaj, jer ga postavlja hardver na kraju obrade i ostaje postavljen dok softver u njega ne upiše jedinicu (**write-1-to-clear**).

Registar **WIDTH** zadržan je radi kompatibilnosti sa referentnim softverskim modelom, ali ga datapath ne koristi, jer je row banking uklonio množenje širinom iz adresne putanje, pa hardveru širina slike više nije potrebna za računanje adrese.

Ograničenja integracije

Dizajn ima dva ograničenja koja nisu vidljiva iz same registarske mape, a čije ignorisanje proizvodi teško uočljive greške.

Prvo, bram_clk_a mora biti isti takt kao s00_axi_aclk. Row banke su taktovane taktom AXI protokola, a ulaz **bram_clk_a** je namjerno ostavljen nepovezan unutar jezgra. Kada bi BRAM port u blok dizajnu bio taktovan drugim taktom, podaci bi prelazili granicu taktnih domena bez sinhronizacije, a rezultat toga ne bi bio očigledan otkaz sistema nego povremeno

pogrešni pikseli, što je vrsta greške koju je vrlo teško uočiti. U blok dizajnu su oba interfejsa taktovana sa **FCLK_CLK0**.

Drugo, softver ne smije upisivati redove dok je jezgro zauzeto. Port A svake banke dijeljen je između PS write puta i jezgrovog čitanja nulte kolone, bez ikakve arbitraže, Dizajn se potpuno oslanja na pretpostavku da se ta dva pristupa nikada ne preklapaju, odnosno da se redovi upisuju isključivo nakon što jezgro postavi **DONE**. Ovo ograničenje je u ovom sistemu preneseno u kernel drajver, koji upis odbija sa greškom **-EBUSY** ako je **BUSY** bit postavljen, čime se pretpostavka iz RTL-a pretvara u pravilo koje sistem aktivno provodi.

Tok podataka kroz sistem

Cjelokupan tok obrade može se podijeliti u nekoliko faza izvršavanja koje će biti opisane u nastavku.

U pripremnoj (nultoj) fazi, Python skripta na host računaru **pretvara ulaznu fotografiju u greyscale PGM format i po potrebi je smanjuje na najviše 360 piksela širine, što je najšira skala koju row banke mogu primiti**. Gotov PGM fajl se zatim prenosi na ploču.

U prvoj fazi, **aplikacija za svaku skalu piramide slika skalira sliku i računa integralnu i kvadriranu integralnu sliku**. Hardverski prozor je fiksne veličine 24 * 24, pa se detekcija lica različitih veličina postiže izgradnjom piramide slika.

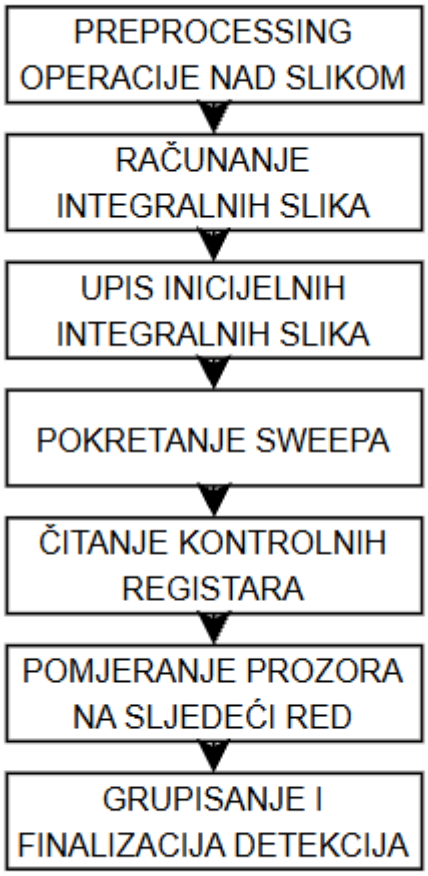
U drugoj fazi, **aplikacija upisuje početni skup od 25 redova integralne slike i dva reda kvadrirane integralne slike u row banke**, kroz **/dev/vj_rows** device fajl.

U trećoj fazi, **aplikacija jednim upisom u /dev/vj_ctrl postavlja parametre pretrage i pokreće sweep po cijelom redu**. Jezgro autonomno testira sve pozicije prozora od X_START do X_LAST sa korakom X_STEP i upisuje pozicije prihvaćenih prozora u hit bafer.

U četvrtoj fazi, **aplikacija čita /dev/vj_ctrl, što blokira dok kernel drajver ne primi DONE prekid**. Kada se čitanje vrati, ono nosi i broj detekcija, pa aplikacija otvara /dev/vj_hits samo ako je taj broj različit od nule.

U petoj fazi, **aplikacija pomjera prozor na sljedeći red**. Zahvaljujući rotaciji ROW_BASE registra, ovo zahtijeva upis samo jednog novog reda integralne slike i dva reda kvadrirane, umjesto ponovnog prenosa cijelog bloka. Postupak se ponavlja od treće faze.

Nakon obrade svih skala, **aplikacija grupiše preklapajuće detekcije**. Isto lice biva prihvaćeno na više susjednih pozicija i na više skala, pa se slični pravougaonici spajaju, a grupe sa premalo članova odbacuju kao lažno pozitivni slučajevi.



Slika 2. Faze rada sistema

Linux kernel drajver

Uloga drajvera u sistemu

Linux kernel drajver je sloj koji premošćuje jaz između korisničkog prostora i hardvera. U Linuxu korisnička aplikacija ne može direktno pristupati fizičkim adresama perifernih uređaja - taj pristup blokira **MMU (Memory Management Unit)**, sastavni dio svake moderne procesorske arhitekture.Rješenje ovog problema je kreiranje **character device drajvera** koji se izvršava u privilegovanom kernel prostoru i ima ovlaštenje da pristupa fizičkim adresama kroz **ioremap()** mehanizam. Korisnička aplikacija komunicira sa drajverom isključivo kroz standardne Unix sistemske pozive **open(), read(), write(), pread(), pwrite() i close()** nad device fajlovima, bez ikakvog znanja o fizičkim adresama ili AXI protokolu.

Pored posredovanja u pristupu, drajver u ovom sistemu preuzima i tri odgovornosti koje aplikacija ne bi mogla pouzdano obaviti, a to su obrada prekida, primjena ograničenja iz RTL-a i sinhronizacija.

Platform drajver i stablo uređaja (device tree)

Drajver je implementiran kao platformski drajver, što je standardni mehanizam za uređaje koji se ne mogu automatski otkriti na magistrali, nego su opisani u device tree-u. Kernel prilikom podizanja sistema prolazi kroz device tree i za svaki node traži drajver čije se **compatible** polje poklapa; kada ga nađe, poziva njegovu **probe()** funkciju i prosljeđuje joj adresne i prekidne resurse iz stabla.

PetaLinux automatski generiše nodeove za oba uređaja iz programabilne logike i svakom dodjeljuje compatible polje izvedeno iz naziva IP jezgra.Postoje dva načina da se drajver upari sa tim nodeovima.

Korištenje automatski generisanog polja je jednostavnije, jer je dovoljno taj string navesti u tabeli uparivanja drajvera i nikakva izmjena stabla uređaja nije potrebna.

Redefinisanje polja kroz **system-user.dtsi** datoteku daje čitljivije nazive i, što je važnije, neophodno je za BRAM kontroler.

Drajver zbog toga u tabeli uparivanja navodi oba stringa po uređaju, pa radi ispravno bez obzira na to koji je pristup izabran:

```
static struct of_device_id vj_of_match[] =
{
    { .compatible = "vj_cascade",           },  /*redefinisano u dtsi*/
    { .compatible = "xlnx,vj-cascade-1.0",  },  /*automatski generisano*/
    { .compatible = "vj_row_bram",          },  /*redefinisano u dtsi*/
    { .compatible = "xlnx,axi-bram-ctrl-4.1", }, /*automatski generisano*/
    { /* end of list */ },
};
```

Listing 1. Tabela uparivanja uređaja

Nazivi nodeova u redefiniciji u tabeli moraju se tačno poklopiti sa onima iz automatski generisanog **pl.dtsi** fajla, koje treba provjeriti prije build-a komandom **petalinux-config -c device-tree**.Interrupt nam takođe dolazi iz istog nodea. Signal **irq** vezan je na **IRQ_F2P[0],** čemu u generisanom stablu odgovara **interrupts = <0 29 4>**.

Prepoznavanje uređaja u probe funkciji

Jedan drajver upravlja sa dva fizički odvojena uređaja, pa **probe()** mora znati koji je od njih u pitanju pri svakom pozivu.

U ovom slučaju se uređaj prepoznaje po **compatible** stringu:

```
static int vj_is_core(struct platform_device *pdev)
{
    return of_device_is_compatible(pdev->dev.of_node, "vj_cascade") ||
           of_device_is_compatible(pdev->dev.of_node, "xlnx,vj-cascade-
1.0");
}

static int vj_is_rows(struct platform_device *pdev)
{
    return of_device_is_compatible(pdev->dev.of_node, "vj_row_bram") ||
           of_device_is_compatible(pdev->dev.of_node, "xlnx,axi-bram-ctrl-
4.1");
}
```

Listing 2. Pomoćne funkcije za prepoznavanje uređaja

Redoslijed kojim kernel prolazi kroz stablo uređaja time prestaje da bude bitan. Za svaki uređaj se izvršava isti niz koraka: dobavljanje adresnog resursa preko **platform_get_resource(),** rezervacija regiona sa **request_mem_region()** i mapiranje sa **ioremap().** Funkcija **ioremap()** ne alocira fizičku memoriju - ona već postoji u obliku hardverskih registara i BRAMa na FPGA - nego samo kreira mapiranje u virtuelnom adresnom prostoru kernela, sa atributima keširanja koji su prilagođeni memorijski mapiranom ulazu i izlazu.

Samo node kontrolnih registara prenosi prekidni resurs, pa se **platform_get_irq()** i **request_irq()** pozivaju samo za njega. Nakon uspješnog mapiranja, jezgro se dovodi u poznato stanje upisom **RESET** bita i brisanjem eventualno zaostalog **DONE** flaga.Ukoliko bilo koji korak ne uspije, resursi se oslobađaju u obrnutom redoslijedu.

Uloga device fajlova

Drajver ima tri **device fajla** koji dijele isti major broj, a razlikuju se po minor broju:

Minor	Device fajl	Namjena
0	/dev/vj_ctrl	Konfiguracija, pokretanje, čekanje na kraj obrade
1	/dev/vj_rows	Binarni upis redova integralne slike

2	/dev/vj_hits	Čitanje liste detekcija

Tabela 3. Device fajlovi drajvera

Registri i podaci o slici imaju bitno različite obrasce pristupa pa im odgovaraju i različiti interfejsi prema korisničkom prostoru.Funkcije **vj_read()** i **vj_write()** određuju o kom uređaju je riječ preko minor broja dobijenog iz **inode** strukture i proslijeđuju poziv odgovarajućoj logici.

Pristup hardveru serijalizovan je **mutexom**, pa dva procesa ne mogu istovremeno pokrenuti obradu ili upisivati redove.

Upis u kontrolni uređaj

Upis u **/dev/vj_ctrl** prihvata razmakom razdvojene tokene, tako da se cijelo podešavanje i pokretanje jednog sweep-a šalje jednim sistemskim pozivom.Podržani su tokeni **reset**, **clear_done**, **start**, **sweep**, te dodjele **x_start=, x_last=, x_step=, width=, row_base=** i **hit_addr=.**

```
static int vj_exec_token(const char *tok)

{
    unsigned int val;

    if (!strcmp(tok, "reset"))
    {
        reg_write(VJ_CTRL, CTRL_RESET);
        vj_done_flag = 0;
        vj_run_pending = 0;
        return 0;
    }
    if (!strcmp(tok, "clear_done"))
    {
        reg_write(VJ_CTRL, CTRL_DONE);
        vj_done_flag = 0;
        return 0;
    }
    if (!strcmp(tok, "start") || !strcmp(tok, "sweep"))
    {
        u32 ctrl = CTRL_START;

        if (reg_read(VJ_STATUS) & STATUS_BUSY)
        {
            printk(KERN_WARNING "vj_driver: start odbijen, jezgro je
zauzeto\n");
            return -EBUSY;
        }
        if (!strcmp(tok, "sweep"))
            ctrl |= CTRL_MODE_SWEEP;

        vj_done_flag   = 0;
        vj_run_pending = 1;
        /*Mode bit se salje zajedno sa START-om jer ga svaki prethodni upis
u CTRL (ukljucujuci brisanje DONE u ISR-u) obrise*/
        reg_write(VJ_CTRL, ctrl);
        return 0;
    }
```

```
    if (sscanf(tok, "x_start=%u", &val) == 1)
    {
        reg_write(VJ_X_START, val);
        return 0;
    }
    if (sscanf(tok, "x_last=%u", &val) == 1)
    {
        reg_write(VJ_X_LAST, val);
        return 0;
    }
    if (sscanf(tok, "x_step=%u", &val) == 1)
    {
        if (val == 0)
        {
            printk(KERN_WARNING "vj_driver: x_step=0 bi zaglavio sweep\n");
            return -EINVAL;
        }
        reg_write(VJ_X_STEP, val);
        return 0;
    }
    if (sscanf(tok, "width=%u", &val) == 1)
    {
        reg_write(VJ_WIDTH, val);
        return 0;
    }
    if (sscanf(tok, "row_base=%u", &val) == 1)
    {
        if (val > 24) return -EINVAL;
        reg_write(VJ_ROW_BASE, val);
        return 0;
    }
    if (sscanf(tok, "hit_addr=%u", &val) == 1)
    {
        if (val >= HIT_DEPTH) return -EINVAL;
        reg_write(VJ_HIT_ADDR, val);
        return 0;
    }

    printk(KERN_WARNING "vj_driver: nepoznata komanda '%s'\n", tok);
    return -EINVAL;
}

static ssize_t vj_ctrl_write(const char __user *buf, size_t len)
{
    char *kbuf, *p, *tok;
    int rc = 0;

    if (len == 0 || len >= CMD_BUF_SIZE)
        return -EINVAL;

    kbuf = kmalloc(len + 1, GFP_KERNEL);
    if (!kbuf)
        return -ENOMEM;

    if (copy_from_user(kbuf, buf, len))
    {
        kfree(kbuf);
        return -EFAULT;
    }
    kbuf[len] = '\0';
```

```
    p = kbuf;
    while ((tok = strsep(&p, " \t\n\r")) != NULL)
    {
        if (*tok == '\0')
            continue;                   /*preskoci visestruke separatore*/
        rc = vj_exec_token(tok);
        if (rc)
            break;
    }

    kfree(kbuf);
    return rc ? rc : (ssize_t)len;
}
```

Listing 3. Funkcije za upis u CTRL registar

Drajver na ovom mjestu provodi i validaciju koju hardver ne radi. Vrijednost x_step=0 bi zaglavila sweep u beskonačnoj petlji, a row_base veći od 24 bi proizveo nevalidnu rotaciju, pa se oba odbijaju sa **-EINVAL**. Pokretanje dok je BUSY postavljen odbija se sa **-EBUSY.**

U registarskom fajlu u RTL-u, upis u CTRL registar bezuslovno dodjeljuje **reg_mode <= S_AXI_WDATA(2)** - to polje nema enable signal , što praktično znači da **svaki upis u CTRL registar briše MODE bit**, uključujući i upis kojim se briše DONE flag na kraju obrade.

Drajver zbog toga nikada ne konfiguriše režim odvojeno, nego MODE bit šalje zajedno sa START bitom u istoj riječi.Da je režim postavljen zasebnim upisom prije pokretanja, brisanje DONE flaga iz prethodne obrade vratilo bi jezgro u režim jednog prozora, a sweep bi vratio tačno jednu detekciju umjesto cijelog reda.

Prekidna rutina

Signal **irq** u RTL-u vezan je direktno na **reg_done** i aktivan je na nivo. Prekidna rutina zbog toga mora obrisati DONE flag prije izlaska, inače bi se prekid odmah ponovo aktivirao i sistem bi ušao u beskonačnu petlju prekida.

```
static irqreturn_t vj_isr(int irq, void *dev_id)
{
    u32 ctrl = reg_read(VJ_CTRL);

    if (!(ctrl & CTRL_DONE))
        return IRQ_NONE;    /*nije nas prekid*/

    /* Rezultate uzimamo dok su registri jos validni */
    vj_last_result = reg_read(VJ_RESULT);
    vj_last_hits   = reg_read(VJ_HIT_COUNT);

    reg_write(VJ_CTRL, CTRL_DONE);      /* write-1-to-clear, spusta irq */

    vj_done_flag   = 1;
    vj_run_pending = 0;
    wake_up_interruptible(&vj_wq);

    return IRQ_HANDLED;
```

```
}
```

Listing 4. Prekidna rutina

Rezultat i broj detekcija se očitavaju u samoj rutini, dok su registri sigurno validni, i pamte se u strukturi drajvera. Time kasnije čitanje iz korisničkog prostora ne zavisi od toga da li je u međuvremenu pokrenuta nova obrada. Brisanje DONE flaga je ovdje dovoljno jer je u RTL-u **irq <= reg_done**, a reg_done se briše upisom jedinice u CTRL **bit4** - dakle operacija kojom rutina potvrđuje prijem prekida istovremeno je i operacija koja spušta prekidnu liniju.

Čitanje iz kontrolnog uređaja

Čitanje iz **/dev/vj_ctrl** blokira ukoliko je obrada u toku, a odmah vraća stanje ukoliko nije:

```
static ssize_t vj_ctrl_read(struct file *f, char __user *buf,
                            size_t len, loff_t *off)
{
    char kbuf[STAT_BUF_SIZE];
    int n, rc;
    u32 status, ctrl;

    if (*off > 0)
        return 0;   /* EOF */

    if (vj_run_pending)
    {
        rc = wait_event_interruptible_timeout(
                 vj_wq, vj_done_flag != 0,
                 msecs_to_jiffies(VJ_DONE_TIMEOUT_MS));

        if (rc == 0)
        {
            printk(KERN_ERR "vj_driver: timeout, DONE prekid nije stigao "
                   "(da li je bitstream ucitan?)\n");
            vj_run_pending = 0;
            return -ETIMEDOUT;
        }
        if (rc < 0)
            return -ERESTARTSYS;    /* prekinuto signalom */
    }

    status = reg_read(VJ_STATUS);
    ctrl   = reg_read(VJ_CTRL);

    n = scnprintf(kbuf, sizeof(kbuf),
                  "busy=%u done=%u result=%d hits=%u\n",
                  status & STATUS_BUSY,
                  vj_done_flag,
                  (int)vj_last_result,
                  vj_last_hits);

    if (len < (size_t)n)
        return -EINVAL;
    if (copy_to_user(buf, kbuf, n))
        return -EFAULT;

    *off += n;
```

```
    return n;
}
```

Listing 5. Funkcija za čitanje iz CTRL registra

Proces se uspavljuje i ne troši procesorsko vrijeme dok čeka jer ga budi prekidna rutina. Timeout od 5 sekundi je konzervativan i postoji zbog jednog konkretnog scenarija: ako bitstream nije učitan na FPGA, na adresi 0x4000_0000 nema hardvera koji bi odgovorio, prekid nikada ne stiže, a bez timeout-a bi proces ostao trajno zaglavljen. Uz timeout, aplikacija dobija jasnu grešku.

Odgovor koji čitanje vraća sadrži i broj detekcija,pa zahvaljujući tome aplikacija otvara /dev/vj_hits samo kada je broj detekcija različit od nule. Pošto većina pozicija reda ne sadrži lice, ovim se izbjegava najveći dio transfera prema hit baferu.

Upis redova

/dev/vj_rows je jedini interfejs velikog obima u sistemu i njegov dizajn direktno određuje ukupne performanse.Interfejs je binarni i pozicioni: pozicija u fajlu jeste bajt offset u BRAM prozoru. Aplikacija zbog toga jednim pozivom prenosi cijeli red.

```
static ssize_t vj_rows_write(const char __user *buf, size_t len, loff_t
*off)
{
    u32 *kbuf;
    size_t done = 0;

    if (!vj_rows)
        return -ENODEV;

    if ((len & 3) || (*off & 3))
    {
        printk(KERN_WARNING "vj_driver: duzina i offset moraju biti umnozak
4\n");
        return -EINVAL;
    }
    if (*off + len > ROW_BRAM_BYTES)
        return -EINVAL;
    if (len == 0)
        return 0;

    if (reg_read(VJ_STATUS) & STATUS_BUSY)
    {
        printk(KERN_WARNING "vj_driver: upis redova odbijen, jezgro je
zauzeto\n");
        return -EBUSY;
    }

    kbuf = kmalloc(ROW_CHUNK, GFP_KERNEL);
    if (!kbuf)
        return -ENOMEM;

    while (done < len)
    {
        size_t chunk = min((size_t)ROW_CHUNK, len - done);
        size_t i;
```

```
        if (copy_from_user(kbuf, buf + done, chunk))
        {
            kfree(kbuf);
            return -EFAULT;
        }
        for (i = 0; i < chunk / 4; i++)
            iowrite32(kbuf[i], vj_rows->base_addr + *off + done + i * 4);

        done += chunk;
    }

    kfree(kbuf);
    *off += len;
    return len;
}
```

Listing 6. Funkcija za upis u row banke

Konkretna posljedica ovog dizajna vidi se pri učitavanju prve pozicije reda na novoj skali. Pošto su banke susjedne u adresnom prostoru, svih 25 banaka integralne slike upisuje se jednim pozivom od 51 200 bajtova umjesto sa 25 zasebnih. Podaci se prenose u batchevima od 4096 bajtova preko **kmalloc()** bafera, umjesto kroz niz promjenljive dužine koji bi na kernel stacku bio opasan. Upis u hardver koristi **iowrite32(),** koji garantuje da se pristupi dese tačno u zadatom redoslijedu, bez preuređivanja od strane kompajlera ili procesora.

Čitanje detekcija

Hit bafer se ne može mapirati u memoriju jer se čita indirektno, kroz registarski par upisom indeksa u **HIT_ADDR** i čitanjem **HIT_DATA**. Drajver taj postupak sakriva od aplikacije tako što pri prvom čitanju prolazi kroz sve validne stavke i formira tekstualnu listu x koordinata, jednu po liniji.

```
static ssize_t vj_hits_read(struct file *f, char __user *buf,
                            size_t len, loff_t *off)
{
    size_t remaining;

    if (*off == 0)
    {
        u32 count, i;

        if (reg_read(VJ_STATUS) & STATUS_BUSY)
            return -EBUSY;

        count = reg_read(VJ_HIT_COUNT);
        if (count > HIT_DEPTH)
        {
            printk(KERN_WARNING "vj_driver: hit_count=%u van opsega\n",
count);
            count = HIT_DEPTH;
        }

        vj_hits_len = 0;
        for (i = 0; i < count; i++)
        {
            u32 x;
```

```
            reg_write(VJ_HIT_ADDR, i);
            x = reg_read(VJ_HIT_DATA);
            vj_hits_len += scnprintf(vj_hits_buf + vj_hits_len,
                                     HITS_BUF_SIZE - vj_hits_len,
                                     "%u\n", x);
        }
    }

    if (*off >= (loff_t)vj_hits_len)
        return 0;    /* EOF */

    remaining = vj_hits_len - *off;
    if (len > remaining)
        len = remaining;

    if (copy_to_user(buf, vj_hits_buf + *off, len))
        return -EFAULT;

    *off += len;
    return len;
}
```

Listing 7. Funkcija za čitanje iz hit bafera

Broj stavki se ograničava na **HIT_DEPTH** i prije korištenja, iako hardver po konstrukciji ne može prijaviti više. Provjera štiti od upisa izvan bafera u slučaju da se čita sa nekonfigurisanog FPGA-a, kada registri vraćaju proizvoljne vrijednosti.

Uklanjanje drajvera

Po završetku rada drajver se uklanja komandom **rmmod**, koja poziva izlaznu funkciju. Ona odjavljuje platform drajver, uklanja character device, uništava device fajlove i klasu uređaja te oslobađa alocirane bafere. Prilikom uklanjanja uređaja sa kontrolnim registrima, jezgro se prvo dovodi u reset stanje, zatim se oslobađa prekid pozivom **free_irq(),** pa tek onda poništava mapiranje sa **iounmap().**

Redoslijed je bitan i obrnut je od redoslijeda inicijalizacije. Kada bi se **iounmap()** pozvao prije **free_irq(),** prekid koji bi u međuvremenu stigao pozvao bi rutinu koja pristupa već poništenom mapiranju, što bi izazvalo crash kernela.

Korisnička aplikacija

Način pokretanja

Korisnička aplikacija je najviši sloj softverskog steka i jedina komponenta sa kojom korisnik direktno interaguje.

Pokreće se sa putanjom do slike i opcionim parametrima. Uz svaki parametar data je i njegova podrazumijevana vrijednost:

```
./vj_app slika.pgm -f 1.2 -x 1 -y 1 -n 3 -o rezultat.txt
```

Opcija	Značenje
-f	Faktor skaliranja piramide slika
-x, -y	Korak pomjeranja prozora po x i y
-n	Minimalan broj susjeda pri grupisanju
-o	Fajl u koji se upisuju rezultati

Tabela 4. Argumenti komandne linije kod pokretanja aplikacije

Priprema slike

Ulazna slika se priprema za obradu Python skriptom na host računaru. Skripta pretvara fotografiju u grayscale i po potrebi je smanjuje na najviše 360 piksela širine, što je najšira skala koju row banke mogu primiti, te je snima u PGM format. PGM je izabran umjesto tekstualnog formata sa jednim brojem po liniji jer je samoopisujući - zaglavlje nosi dimenzije, pa aplikacija ne mora pretpostavljati veličinu slike.

```
def cmd_prepare(args):
    img = cv2.imread(args.source, cv2.IMREAD_GRAYSCALE)
    if img is None:
        sys.exit(f"ne mogu procitati '{args.source}'")

    h, w = img.shape

    if w > args.max_width:
        # Sirina preko MAX_IMG_WIDTH znaci da najsire skale ne staju u
        # row banke, pa se slika smanjuje jos na hostu.
        scale = args.max_width / w
        new_w = args.max_width
        new_h = max(1, int(round(h * scale)))
        img = cv2.resize(img, (new_w, new_h), interpolation=cv2.INTER_AREA)
        print(f"smanjeno {w}x{h} -> {new_w}x{new_h} (MAX_IMG_WIDTH =
{args.max_width})")
        h, w = img.shape

    # PGM P5 je binaran i opisuje samog sebe, pa aplikacija ne mora
```

```
    # pogadjati dimenzije.
    cv2.imwrite(args.dest, img)
    print(f"upisano {args.dest}: {w}x{h}, {w * h} piksela")

    # Kontrolni ispis:koliko skala hardver zaista obradi.
    scale = 1.0
    n = 0
    while True:
        sw, sh = int(round(w / scale)), int(round(h / scale))
        if sw < 24 or sh < 24:
            break
        if sw + 1 <= 512:
            n += 1
        scale *= args.factor
    print(f"pri faktoru {args.factor} bice obradjeno {n} skala")
```

Listing 8. Dio Python skripte za preprocessing operacije

Integralna i kvadrirana integralna slika

Za svaku skalu aplikacija računa integralnu i kvadriranu integralnu sliku dimenzija (h+1) * (w+1), gdje element sadrži sumu svih piksela sa redom manjim od r i kolonom manjom od c:

```
static void integral_first(const image_t *im, int32_t *sum, int32_t *sqsum,
                           int width, int height)
{
    int x_ratio = (int)(((int64_t)im->w << 16) / width) + 1;
    int y_ratio = (int)(((int64_t)im->h << 16) / height) + 1;
    int i, j;

    for (i = 0; i < INT_ROWS; i++)
    {
        int src_i = (i * y_ratio) >> 16;
        int rat = 0, s = 0, sq = 0;

        if (src_i >= im->h) src_i = im->h - 1;
        for (j = 0; j < width; j++)
        {
            int src_j = rat >> 16;
            int pix;
            int32_t above, above_sq;

            if (src_j >= im->w) src_j = im->w - 1;
            rat += x_ratio;
            pix = im->pix[src_i * im->w + src_j];
            s += pix;
            sq += pix * pix;
            above    = i ? sum[(i - 1) * width + j]    : 0;
            above_sq = i ? sqsum[(i - 1) * width + j] : 0;
            sum[i * width + j]   = s + above;
            sqsum[i * width + j] = sq + above_sq;
        }
    }
}

static void integral_lastrow(const image_t *im, int32_t *sum, int32_t
*sqsum,
                             int width, int sz_height, int y_bias)
{
```

```
    int x_ratio = (int)(((int64_t)im->w << 16) / width) + 1;
    int y_ratio = (int)(((int64_t)im->h << 16) / sz_height) + 1;
    int src_i, rat = 0, s = 0, sq = 0, x, y;

    /* Lokalne kopije se pomjeraju za jedan red */
    for (y = 0; y < INT_ROWS - 1; y++)
    {
        memmove(&sum[y * width],   &sum[(y + 1) * width],   width *
sizeof(int32_t));
        memmove(&sqsum[y * width], &sqsum[(y + 1) * width], width *
sizeof(int32_t));
    }

    src_i = ((WIN_SIZE + y_bias) * y_ratio) >> 16;
    if (src_i >= im->h) src_i = im->h - 1;

    for (x = 0; x < width; x++)
    {
        int src_j = rat >> 16, pix;
        if (src_j >= im->w) src_j = im->w - 1;
        rat += x_ratio;
        pix = im->pix[src_i * im->w + src_j];
        s += pix;
        sq += pix * pix;
        sum[24 * width + x]   = s  + sum[23 * width + x];
        sqsum[24 * width + x] = sq + sqsum[23 * width + x];
    }
}
```

Listing 9. Funkcije za rad sa integralnom slikom

Pomjeranje prozora po y

Ovo je mjesto na kojem se rotacija banaka pretvara u kod. Novi red se upisuje u banku sa indeksom stare baze, a tek zatim se baza inkrementira. Redoslijed ova dva koraka nije zamjenjiv. Zajedno s novim redom integralne slike, ažuriraju se i vrijednosti redova u dvije banke za kvadriranu integralnu sliku.

```
/*Pomjeranje prozora po y: jedan novi red + dva kvadrirana*/
        for (y_bias = 1; y_bias < sz_h - INT_ROWS + 1; y_bias++)
        {
            integral_lastrow(im, g_int, g_sq, sz_w, sz_h, y_bias);

            if (vj_upload_row(d, row_base, &g_int[24 * sz_w], sz_w)) goto
fail;
            row_base = (row_base + 1) % INT_ROWS;
            if (vj_upload_row(d, SQ0_BANK,  &g_sq[0  * sz_w], sz_w)) goto
fail;
            if (vj_upload_row(d, SQ23_BANK, &g_sq[23 * sz_w], sz_w)) goto
fail;

            got = vj_sweep(d, row_base, sz_w, x_last, shift_step, xs,
HIT_DEPTH);
            if (got < 0) goto fail;
            for (k = 0; k < got && nf < max_faces; k++)
            {
                faces[nf].x = my_round(xs[k] * factor);
                faces[nf].y = my_round(y_bias * factor);
```

```
                faces[nf].w = win_w;
                faces[nf].h = win_h;
                nf++;
            }
            if (verbose && got)
                printf("[APP]   skala %.4f  y=%3d: %d detekcija\n", factor,
y_bias, got);
        }
```

Listing 10. Prikaz koda za pomjeranje prozora

Pokretanje sweep-a i čekanje

Za svaku poziciju reda aplikacija jednim upisom postavlja parametre i pokreće obradu, a zatim čeka na povratnu informaciju.Funkcija **vj_wait_done()** čita **/dev/vj_ctrl** pozivom **pread()** sa offsetom nula, čime se svaki put ponovo pokreće blokirajuće čitanje. Callback se dešava tek kada drajver primi **DONE** prekid, i prenosi broj detekcija. Detekcije se čitaju samo ako je broj različit od nule, nakon čega se koordinate iz prostora skale prevode nazad u koordinate originalne slike množenjem faktorom skale.

```
static int vj_sweep(vj_dev_t *d, int row_base, int width, int x_last, int
step,
                    int *xs, int max)
{
    int hits, got;

    if (vj_cmd(d, "row_base=%d width=%d x_start=0 x_last=%d x_step=%d
sweep",
               row_base, width, x_last, step))
        return -1;
    stat_sweeps++;

    hits = vj_wait_done(d);
    if (hits < 0) return -1;
    if (hits == 0) return 0;

    got = vj_read_hits(d, xs, max);
    if (got < 0) return -1;
    stat_raw_hits += got;
    return got;
}
```

Listing 11. Prikaz funkcije za sweep pretragu

Grupisanje detekcija

Hardver prijavljuje svaki prihvaćeni prozor, pa zbog toga jedno lice proizvodi detekcije na više susjednih pozicija i na više skala, pa sirovi izlaz sadrži znatno više pravougaonika nego što ima lica. Aplikacija primjenjuje standardni postupak grupisanja: pravougaonici se smatraju sličnim ako se sve četiri njihove ivice razlikuju za manje od praga proporcionalnog njihovoj veličini, slični se spajaju u grupu, a grupa se prihvata samo ako ima najmanje -n članova. Rezultat je aritmetička sredina članova grupe.

Pri pokretanju sistema korisno je znati da veliki broj sirovih detekcija uz nula grupisanih ne znači problem u hardveru, nego prestrog prag -n.

```
static int group_rectangles(rect_t *r, int n, int group_threshold, double
eps)
{
    int labels[MAX_FACES], cnt[MAX_FACES];
    rect_t acc[MAX_FACES];
    int nclasses, i, j, out = 0;

    if (group_threshold <= 0 || n == 0) return n;
    nclasses = partition_rects(r, n, labels, eps);

    for (i = 0; i < nclasses; i++)
    {
        acc[i].x = acc[i].y = acc[i].w = acc[i].h = 0;
        cnt[i] = 0;
    }
    for (i = 0; i < n; i++)
    {
        int c = labels[i];
        acc[c].x += r[i].x;
        acc[c].y += r[i].y;
        acc[c].w += r[i].w;
        acc[c].h += r[i].h;
        cnt[c]++;
    }
    for (i = 0; i < nclasses; i++)
    {
        double s = q_ufixed(1.0 / cnt[i], 10, 1);
        acc[i].x = my_round(acc[i].x * s);
        acc[i].y = my_round(acc[i].y * s);
        acc[i].w = my_round(acc[i].w * s);
        acc[i].h = my_round(acc[i].h * s);
    }
    for (i = 0; i < nclasses; i++)
    {
        int n1 = cnt[i];
        if (n1 <= group_threshold) continue;
        for (j = 0; j < nclasses; j++)
        {
            int n2 = cnt[j], dx, dy;
            if (j == i) continue;
            if (n2 <= group_threshold) continue;
            dx = my_round(acc[j].w * eps);
            dy = my_round(acc[j].h * eps);
            if (acc[i].x >= acc[j].x - dx && acc[i].y >= acc[j].y - dy &&
                    acc[i].x + acc[i].w <= acc[j].x + acc[j].w + dx &&
                    acc[i].y + acc[i].h <= acc[j].y + acc[j].h + dy &&
                    (n2 > (3 > n1 ? 3 : n1) || n1 < 3))
                break;
        }
        if (j == nclasses && out < MAX_FACES) r[out++] = acc[i];
    }
    return out;
}
```

Listing 12. Glavna funkcija za grupisanje pravougaonika

Pokretanje sistema i rezultati

Kompajliranje i učitavanje drajvera

Prije prvog kompajliranja modula, na ploči je potrebno buildovati pomoćne alate kernel build sistema. Nakon toga se drajverski modul pravi i učitava standardnim postupkom:

$ make
$ sudo insmod vj_driver.ko

Dodatno, potrebno je i podesiti dozvole za ispravan rad aplikacije:

sudo chmod 666 /dev/vj_*

Rezultati detekcije

Sistem je testiran na fotografiji grupe ljudi (Solvay konferencija) dimenzija 360 * 240 piksela, koja sadrži osam frontalnih lica. Ista slika prethodno je obrađena Vitis baremetal aplikacijom, što je omogućilo direktno poređenje. Rezultat od osam detekcija poklapa se sa brojem lica na fotografiji i sa rezultatom referentne Vitis aplikacije.

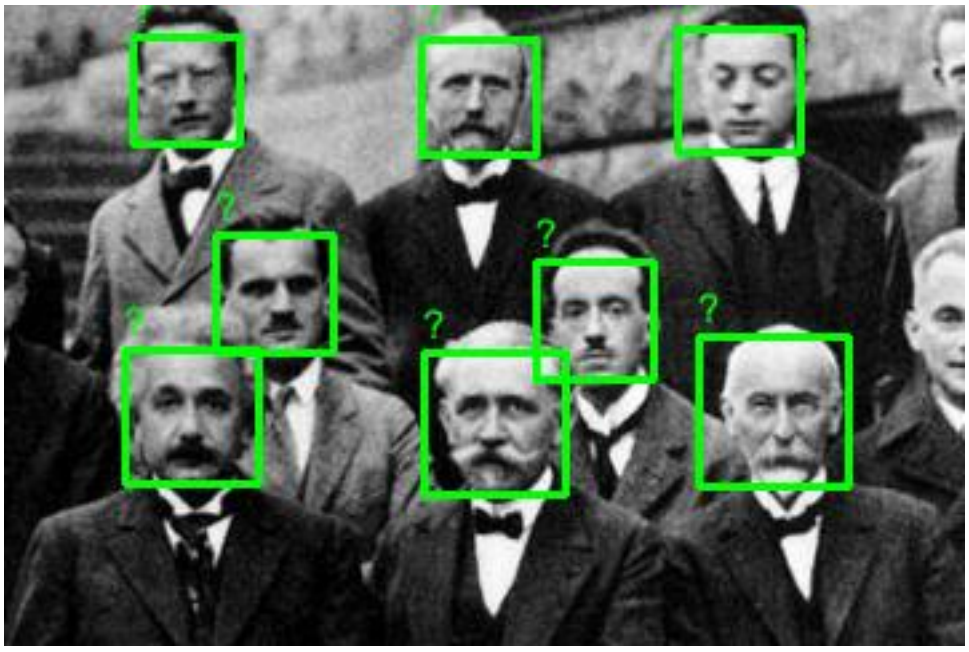
```
vj_linux:~/app$ ./vj_app Face.pgm -o rezultat.txt
[APP] Slika: Face.pgm (360 x 240)
[APP] scaleFactor 1.2031, shiftStep 1, grupisanje prag 1 eps 0.40
[APP] Skala obradjeno: 13

============== REZULTATI ===============
Sweep-ova pokrenuto : 952
Redova poslato      : 3168
Sirovih detekcija   : 97
Nakon grupisanja    : 8
Vrijeme             : 359.78 ms
========================================

Detektovana lica (x y w h):
  lice  0 :  x= 48  y= 12  w= 40  h= 40
  lice  1 :  x=156  y= 13  w= 43  h= 43
  lice  2 :  x= 79  y= 86  w= 44  h= 44
  lice  3 :  x=199  y= 96  w= 44  h= 44
  lice  4 :  x=260  y=124  w= 56  h= 56
  lice  5 :  x=252  y=  9  w= 46  h= 46
  lice  6 :  x= 45  y=129  w= 50  h= 50
  lice  7 :  x=157  y=130  w= 53  h= 53

[APP] Rezultat upisan u rezultat.txt
```

Slika 3. Terminalski ispis nakon pokretanja aplikacije



Slika 4. Slika s označenim licima nakon pokretanja Python skripte

Vizualizacija preko Python skripte potvrđuje da svih osam pravougaonika leži na stvarnim licima. Lica koja detektor nije prijavio su ona djelimično zaklonjena ili odsječena ivicom kadra - dakle upravo one situacije u kojima se od algoritma i ne očekuje detekcija.

Analiza izmjerenih veličina

Ukupno vrijeme od 359.78 ms na 952 pokretanja hardvera daje približno 377.9 mikrosekundi po jednoj poziciji reda. U tom intervalu su sadržani upis tri reda kroz device fajl, jedan upis u kontrolni registar, hardverska obrada cijelog reda slike, prekid i njegova obrada, te buđenje procesa koji je blokiran u **read()** pozivu. Kako sama hardverska obrada jednog reda traje reda veličine desetina mikrosekundi, dominantan dio ovog vremena otpada na softverski sloj - prelaske između korisničkog i kernel prostora i kašnjenje izazvano prekidima.

Zaključak

Kompletan projekat izrade ovakvog embeded sistema demonstrira kompletan tok razvoja koji obuhvata sve slojeve modernog heterogenog računarskog sistema – od razvoja virtuelne platforme visoke apstrakcije u programskom jeziku visokog nivoa, preko RTL opisa hardvera u VHDL jeziku, pa sve do verifikacionog okruženja, generisanja kernel drajvera, korisničke aplikacije i pomoćnih alata. Svaki sloj je razvijen i provjeren zasebno, a zatim integrisan u sistem koji radi na stvarnom hardveru i daje rezultate identične specifikaciji.

Linux kernel drajver ilustruje standardne mehanizme kernel programiranja - platform drajver interfejs sa uparivanjem preko stabla uređaja, **ioremap()** za pristup memorijski mapiranom hardveru, obradu prekida i blokirajuće čekanje umjesto pollinga.

Korisnička aplikacija demonstrira Unix filozofiju u praksi - sva komunikacija sa hardverom odvija se kroz standardne operacije nad fajlovima, bez ikakvog znanja o fizičkim adresama ili AXI protokolu.

Sistem je uspješno testiran na Zybo Z7-10 ploči sa PetaLinux 2022.2 i kernelom 5.15.36, a dobijeni rezultat se poklapa sa rezultatom referentne Vitis baremetal aplikacije.

Literatura

AMD/Xilinx, „Zynq-7000 SoC Technical Reference Manual”, UG585.

AMD/Xilinx, „Vivado Design Suite: AXI Reference Guide”, UG1037.

J. Corbet, A. Rubini, G. Kroah-Hartman, „Linux Device Drivers, Third Edition”, O’Reilly Media, 2005.

Materijali sa kursa Embeded operativni sistemi, Fakultet tehničkih nauka, Novi Sad.

AMD/Xilinx, „PetaLinux Tools Documentation: Reference Guide (UG1144), v2019.2”.

Digilent, „Zybo Z7 Reference Manual”.

P. Viola, M. Jones, „Rapid Object Detection using a Boosted Cascade of Simple Features”, Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR), 2001.

P. Viola, M. Jones, „Robust Real-Time Face Detection”, International Journal of Computer Vision, 2004.